AI Laboratory – Group B Assignments

LA 9 – Breadth First Search Using Python

Title

Implementation of Breadth First Search (BFS) using Python

Aim

To implement the Breadth First Search algorithm using Python for traversing/searching a graph.

Problem Statement

Write a Python program to implement Breadth First Search (BFS) for a given graph. The program should start from a specified source node and visit all reachable nodes level by level.

Prerequisite

- Basic knowledge of Python programming.
- Understanding of graphs and graph representation.
- Knowledge of queues.
- Basic understanding of searching algorithms.

Software Requirements

- Operating System: Windows/Linux/macOS
- Python 3.x
- IDE: VS Code / PyCharm / IDLE / Jupyter Notebook

Learning Objectives

- Understand the concept of uninformed search.
- Learn the working of BFS.
- Understand queue-based graph traversal.
- Implement BFS using Python.

Outcomes

After completing this experiment, students will be able to:

- Explain the working of BFS.

- Represent a graph using Python.
- Implement BFS using a queue.
- Determine the traversal order of a graph.

Theory

Breadth First Search (BFS) is an uninformed search technique that explores a graph **level by level**. It first visits the starting node and then visits all its adjacent nodes before moving to the next level.

BFS uses a **Queue (FIFO – First In, First Out)** data structure.

For an unweighted graph, BFS can also be used to find the shortest path in terms of the number of edges.

Algorithm

1. Start with the source node.
2. Create an empty queue.
3. Mark the source node as visited.
4. Insert the source node into the queue.
5. Repeat until the queue becomes empty:
    - Remove a node from the front of the queue.
    - Visit the node.
    - Add all unvisited adjacent nodes to the queue.
    - Mark each newly added node as visited.
6. Stop when all reachable nodes are visited.

Time Complexity

For a graph represented using an adjacency list:

Time Complexity: O(V + E)

Where:

- V = Number of vertices
- E = Number of edges

Space Complexity: O(V)

Conclusion

The Breadth First Search algorithm was successfully implemented using Python. BFS traverses the graph level by level using a queue.

Viva Questions

1. What is BFS?
2. Which data structure is used in BFS?
3. What does FIFO mean?
4. What is the time complexity of BFS?
5. What is the difference between BFS and DFS?
6. Can BFS find the shortest path?
7. What is an uninformed search?
8. What happens if a graph contains cycles?
9. Why is a visited list required in BFS?
10. What are the applications of BFS?

LA 10 – Depth First Search Using Python

Title

Implementation of Depth First Search (DFS) using Python

Aim

To implement the Depth First Search algorithm using Python for traversing/searching a graph.

Problem Statement

Write a Python program to implement Depth First Search (DFS) for a given graph. The program should start from a specified node and visit the nodes by exploring one branch as deeply as possible before backtracking.

Prerequisite

- Basic Python programming.
- Knowledge of graphs.
- Understanding of stacks and recursion.
- Basic knowledge of search algorithms.

Software Requirements

- Windows/Linux/macOS
- Python 3.x
- VS Code / PyCharm / IDLE / Jupyter Notebook

Learning Objectives

- Understand depth-first graph traversal.
- Learn the use of stack and recursion.
- Understand backtracking.
- Implement DFS using Python.

Outcomes

Students will be able to:

- Explain DFS.
- Implement DFS using recursion or a stack.
- Traverse graphs using DFS.

- Compare BFS and DFS.

Theory

Depth First Search is an uninformed search technique that explores a graph by going as deep as possible along one branch before returning and exploring another branch.

DFS can be implemented using:

- **Stack**
- **Recursion**

Algorithm

1. Select the starting node.
2. Mark the node as visited.
3. Display/process the node.
4. Visit an unvisited adjacent node.
5. Continue recursively until no unvisited adjacent node is available.
6. Backtrack to the previous node.
7. Continue until all reachable nodes are visited.

Time Complexity

For an adjacency-list representation:

Time Complexity: O(V + E)

Space Complexity: O(V)

Conclusion

The Depth First Search algorithm was successfully implemented using Python. DFS explores one path deeply before backtracking to explore other paths.

Viva Questions

1. What is DFS?
2. Which data structure is used by DFS?
3. Can DFS be implemented recursively?
4. What is backtracking?
5. What is the time complexity of DFS?
6. What is the difference between BFS and DFS?

7. What is the role of the visited array?
8. What are the applications of DFS?
9. Can DFS be used for cycle detection?
10. What is the space complexity of DFS?

LA 11 – Tic-Tac-Toe Game Using Python

Title

Implementation of Tic-Tac-Toe Game using Python

Aim

To implement a Tic-Tac-Toe game using Python and apply AI game-playing concepts.

Problem Statement

Write a Python program to implement a two-player Tic-Tac-Toe game. The program should allow players to enter their moves, display the game board, identify the winner, and determine whether the game ends in a draw.

Prerequisite

- Basic Python programming.
- Conditional statements and loops.
- Lists and functions.
- Basic knowledge of game playing in AI.
- Understanding of decision-making.

Software Requirements

- Python 3.x
- VS Code / PyCharm / IDLE / Jupyter Notebook
- Windows/Linux/macOS

Learning Objectives

- Understand game-playing problems.
- Implement a board-based game.
- Apply conditional logic for decision-making.
- Understand the basic concept of Minimax.

Outcomes

Students will be able to:

- Develop a Tic-Tac-Toe game.
- Represent a game board in Python.
- Identify winning and draw conditions.

- Understand basic AI game-playing concepts.

Theory

Tic-Tac-Toe is a two-player, turn-based game played on a 3 × 3 board. One player uses **X** and the other uses **O**.

A player wins when three identical symbols occur:

- In a row
- In a column
- Diagonally

In AI-based Tic-Tac-Toe, the **Minimax algorithm** can be used to select the best possible move.

Algorithm

1. Initialize a 3 × 3 empty board.
2. Assign symbols X and O to the players.
3. Display the board.
4. Ask the current player to enter a valid position.
5. Update the board.
6. Check whether the player has won.
7. If there is no winner, check whether the board is full.
8. If the board is full, declare a draw.
9. Otherwise, switch the player.
10. Repeat until the game ends.

Time Complexity

For a basic Tic-Tac-Toe game, checking the board takes approximately:

Time Complexity: O(1)

For a Minimax-based implementation, the worst-case complexity is approximately:

O(b^d)

where b is the branching factor and d is the depth of the game tree.

Conclusion

The Tic-Tac-Toe game was successfully implemented using Python. The experiment demonstrates game representation, decision-making, and basic AI game-playing concepts.

Viva Questions

1. What is Tic-Tac-Toe?
2. What is a game tree?
3. What is Minimax?
4. What is a terminal state?
5. What is a draw?
6. What is the purpose of a utility function?
7. What is the difference between a human player and an AI player?
8. What is the branching factor?
9. How can Tic-Tac-Toe be made intelligent?
10. What is the role of Minimax in game playing?

LA 12 – 8-Puzzle Problem Using Python

Title

Implementation of 8-Puzzle Problem using Python

Aim

To implement the 8-Puzzle problem using Python and solve it using a suitable search technique.

Problem Statement

Write a Python program to solve the 8-Puzzle problem. The program should transform a given initial configuration into a specified goal configuration by moving the blank tile.

Prerequisite

- Python programming.
- Knowledge of lists and functions.
- Understanding of search algorithms.
- Knowledge of state-space representation.

Software Requirements

- Python 3.x
- VS Code / PyCharm / IDLE / Jupyter Notebook
- Windows/Linux/macOS

Learning Objectives

- Understand state-space search.
- Represent a puzzle as a state.
- Generate possible successor states.
- Apply search techniques to solve the puzzle.

Outcomes

Students will be able to:

- Represent the 8-Puzzle as a state-space problem.
- Generate valid puzzle moves.
- Implement a search algorithm.

- Find a sequence of moves from the initial state to the goal state.

Theory

The 8-Puzzle consists of a 3 × 3 board containing eight numbered tiles and one blank space.

A typical goal state is:

1 2 3

4 5 6

7 8 _

The blank tile can move **up, down, left, or right**, provided the movement is within the board.

The puzzle can be represented using:

- Initial state
- Goal state
- Operators/actions
- State transition
- Path/cost

BFS can be used to find the shortest solution when every move has equal cost.

Algorithm

1. Define the initial state and goal state.
2. Store the initial state in the search structure.
3. Find the position of the blank tile.
4. Generate all valid movements of the blank tile.
5. Create new states for each valid movement.
6. Mark already explored states.
7. Continue searching until the goal state is reached.
8. Display the sequence of moves.

Time Complexity

For BFS:

Time Complexity: O(b^d)

Space Complexity: O(b^d)

where:

- b = branching factor
- d = depth of the solution

Conclusion

The 8-Puzzle problem was successfully represented and solved using a search technique. The experiment demonstrates state-space representation and problem-solving using AI search.

Viva Questions

1. What is the 8-Puzzle problem?
2. What is a state space?
3. What is the initial state?
4. What is the goal state?
5. What are the possible moves in 8-Puzzle?
6. Why is a visited set required?
7. Which search technique can find the shortest solution?
8. What is a heuristic function?
9. What is Manhattan distance?
10. What is the difference between BFS and A* for 8-Puzzle?